

Assignment No. 09

- **TITLE** – In real-world software like music playlists different types of linked lists are used to manage dynamic data efficiently. Implement singly, doubly, circular, and doubly circular linked lists to explore how each structure handles various operations such as create, insert, delete, traverse, search, sort, and concatenate. Simulate practical scenarios (like adding, deleting, updating or rearranging songs to playlist) to understand the strengths and use-cases of each linked list type.

➤ **THEORY** –

Linked lists are dynamic data structures that store elements in nodes, with each node containing data and pointer(s) to other nodes. They excel in scenarios requiring frequent insertions and deletions, making them ideal for music playlist management where users constantly add, remove, and rearrange songs.

Types of Linked Lists

1. Singly Linked List (SLL)

- **Structure:** Each node points only to the next node
- **Memory:** 1 pointer per node
- **Use Case:** Simple sequential playback, memory-efficient applications
- **Limitation:** Can only traverse forward

2. Doubly Linked List (DLL)

- **Structure:** Each node has both next and previous pointers
- **Memory:** 2 pointers per node
- **Use Case:** Bidirectional navigation (next/previous song), frequent editing
- **Advantage:** Efficient backward traversal

3. Circular Linked List (CLL)

- **Structure:** Last node points back to first node, forming a circle
- **Memory:** 1 pointer per node
- **Use Case:** Continuous playback, loop/repeat functionality
- **Advantage:** No end point, continuous traversal

4. Doubly Circular Linked List (DCLL)

- **Structure:** Circular + bidirectional pointers
- **Memory:** 2 pointers per node
- **Use Case:** Advanced music players with bidirectional continuous playback
- **Advantage:** Most versatile navigation

Algorithms for Linked List Operations

1. CREATE/ADD Operation

Singly Linked List:

Algorithm addSong(SLL, title, artist, duration):

1. Create newSong node with given data
2. IF head is NULL:
 head ← newSong
- ELSE:

```

temp ← head
WHILE temp.next ≠ NULL:
    temp ← temp.next
temp.next ← newSong

```

3. END

Doubly Linked List:

Algorithm addSong(DLL, title, artist, duration):

1. Create newSong node with given data
2. IF head is NULL:

```

    head ← newSong

```

ELSE:

```

    temp ← head
    WHILE temp.next ≠ NULL:
        temp ← temp.next
    temp.next ← newSong
    newSong.prev ← temp

```

3. END

Circular Linked List:

Algorithm addSong(CLL, title, artist, duration):

1. Create newSong node with given data
2. IF head is NULL:

```

    head ← newSong
    newSong.next ← head

```

ELSE:

```

    temp ← head
    WHILE temp.next ≠ head:
        temp ← temp.next
    temp.next ← newSong
    newSong.next ← head

```

3. END

2. DELETE Operation

Singly Linked List:

Algorithm deleteSong(SLL, title):

1. IF head is NULL: RETURN
2. IF head.title = title:


```

                temp ← head
                head ← head.next
                DELETE temp
                RETURN
            
```
3. temp ← head
4. WHILE temp.next ≠ NULL AND temp.next.title ≠ title:


```

                temp ← temp.next
            
```
5. IF temp.next ≠ NULL:


```

                toDelete ← temp.next
            
```

```
temp.next ← temp.next.next
DELETE toDelete
```

6. END

Doubly Linked List:

Algorithm deleteSong(DLL, title):

1. temp ← head
2. WHILE temp ≠ NULL AND temp.title ≠ title:


```
temp ← temp.next
```
3. IF temp = NULL: RETURN
4. IF temp.prev ≠ NULL:


```
temp.prev.next ← temp.next
```
- ELSE:


```
head ← temp.next
```
5. IF temp.next ≠ NULL:


```
temp.next.prev ← temp.prev
```
6. DELETE temp
7. END

Circular Linked List:

Algorithm deleteSong(CLL, title):

1. IF head is NULL: RETURN
2. IF head.title = title AND head.next = head:


```
DELETE head
head ← NULL
RETURN
```
3. IF head.title = title:


```
temp ← head
WHILE temp.next ≠ head:
  temp ← temp.next
temp.next ← head.next
DELETE head
head ← temp.next
RETURN
```
4. temp ← head
5. WHILE temp.next ≠ head AND temp.next.title ≠ title:


```
temp ← temp.next
```
6. IF temp.next.title = title:


```
toDelete ← temp.next
temp.next ← temp.next.next
DELETE toDelete
```
7. END

3. SEARCH Operation

Singly Linked List:

Algorithm searchSong(SLL, title):

1. temp ← head

2. WHILE temp $\neq$ NULL:
 - IF temp.title = title: RETURN TRUE
 - temp $\leftarrow$ temp.next
3. RETURN FALSE

Circular Linked List:

Algorithm searchSong(CLL, title):

1. IF head is NULL: RETURN FALSE
2. temp $\leftarrow$ head
3. DO:
 - IF temp.title = title: RETURN TRUE
 - temp $\leftarrow$ temp.next
 - WHILE temp $\neq$ head
4. RETURN FALSE

4. UPDATE Operation**All List Types:**

Algorithm updateSong(list, title, newTitle, newArtist, newDuration):

1. temp $\leftarrow$ head
2. WHILE temp $\neq$ NULL (or temp $\neq$ head for CLL):
 - IF temp.title = title:
 - temp.title $\leftarrow$ newTitle
 - temp.artist $\leftarrow$ newArtist
 - temp.duration $\leftarrow$ newDuration
 - RETURN
 - temp $\leftarrow$ temp.next
3. END

5. TRAVERSE/DISPLAY Operation**Singly Linked List:**

Algorithm displayPlaylist(SLL):

1. temp $\leftarrow$ head
2. WHILE temp $\neq$ NULL:
 - PRINT temp.title, temp.artist, temp.duration
 - temp $\leftarrow$ temp.next
3. END

Circular Linked List:

Algorithm displayPlaylist(CLL):

1. IF head is NULL: RETURN
2. temp $\leftarrow$ head
3. DO:
 - PRINT temp.title, temp.artist, temp.duration
 - temp $\leftarrow$ temp.next
 - WHILE temp $\neq$ head
4. END

6. SORT/ARRANGE Operation**Bubble Sort Algorithm (All Types):**

Algorithm arrangeSongs(list):

1. IF head is NULL: RETURN
2. swapped $\leftarrow$ TRUE
3. WHILE swapped:
 - swapped $\leftarrow$ FALSE
 - current $\leftarrow$ head
 - WHILE current.next $\neq$ NULL (or $\neq$ head for CLL):
 - IF current.title > current.next.title:
 - SWAP(current.data, current.next.data)
 - swapped $\leftarrow$ TRUE
 - current $\leftarrow$ current.next
4. END

7. CONCATENATE Operation

Singly Linked List:

Algorithm concatenate(SLL, otherList):

1. IF head is NULL:
 - head $\leftarrow$ otherList.head
- ELSE:
 - temp $\leftarrow$ head
 - WHILE temp.next $\neq$ NULL:
 - temp $\leftarrow$ temp.next
 - temp.next $\leftarrow$ otherList.head
2. otherList.head $\leftarrow$ NULL
3. END

Time Complexity Analysis

All operations: **O(n)** for search/delete/update in worst case

- **Best Case:** O(1) when operating on head
- **Average Case:** O(n) for random operations
- **Worst Case:** O(n) when element is at end

➤ CODE –

Music_player.cpp

```
#include <iostream>
#include <string>
using namespace std;
class Song {
public:
    string title;
    string artist;
    int duration;
    Song* next;
    Song* prev;

    Song(string t, string a, int d) {
        title = t;
```

```

        artist = a;
        duration = d;
        next = nullptr;
        prev = nullptr;
    }
};

class SinglyLinkedList {
private:
    Song* head;
public:
    SinglyLinkedList() {
        head = nullptr;
    }
    void addSong(string title, string artist, int duration) {
        Song* newSong = new Song(title, artist, duration);
        if (head == nullptr) {
            head = newSong;
        } else {
            Song* temp = head;
            while (temp->next != nullptr) {
                temp = temp->next;
            }
            temp->next = newSong;
        }
    }
    void deleteSong(string title) {
        if (head == nullptr) return;
        if (head->title == title) {
            Song* temp = head;
            head = head->next;
            delete temp;
            return;
        }
        Song* temp = head;
        while (temp->next != nullptr && temp->next->title != title) {
            temp = temp->next;
        }
        if (temp->next != nullptr) {
            Song* toDelete = temp->next;
            temp->next = temp->next->next;
            delete toDelete;
        }
    }
    bool searchSong(string title) {
        Song* temp = head;
        while (temp != nullptr) {
            if (temp->title == title) return true;
            temp = temp->next;
        }
        return false;
    }
};

```

```

void updateSong(string title, string newTitle, string newArtist, int newDuration)
{
    Song* temp = head;
    while (temp != nullptr) {
        if (temp->title == title) {
            temp->title = newTitle;
            temp->artist = newArtist;
            temp->duration = newDuration;
            return;
        }
        temp = temp->next;
    }
}

void displayPlaylist() {
    Song* temp = head;
    while (temp != nullptr) {
        cout << temp->title << " - " << temp->artist << " (" << temp->duration <<
"s)" << endl;
        temp = temp->next;
    }
}

void arrangeSongs() {
    if (head == nullptr) return;
    bool swapped;
    do {
        swapped = false;
        Song* current = head;
        while (current->next != nullptr) {
            if (current->title > current->next->title) {
                swap(current->title, current->next->title);
                swap(current->artist, current->next->artist);
                swap(current->duration, current->next->duration);
                swapped = true;
            }
            current = current->next;
        }
    } while (swapped);
}

};

class DoublyLinkedList {
private:
    Song* head;
public:
    DoublyLinkedList() {
        head = nullptr;
    }
    void addSong(string title, string artist, int duration) {
        Song* newSong = new Song(title, artist, duration);
        if (head == nullptr) {
            head = newSong;
        } else {
            Song* temp = head;

```

```

        while (temp->next != nullptr) {
            temp = temp->next;
        }
        temp->next = newSong;
        newSong->prev = temp;
    }
}

void deleteSong(string title) {
    if (head == nullptr) return;
    Song* temp = head;
    while (temp != nullptr && temp->title != title) {
        temp = temp->next;
    }
    if (temp == nullptr) return;
    if (temp->prev != nullptr) {
        temp->prev->next = temp->next;
    } else {
        head = temp->next;
    }
    if (temp->next != nullptr) {
        temp->next->prev = temp->prev;
    }
    delete temp;
}

bool searchSong(string title) {
    Song* temp = head;
    while (temp != nullptr) {
        if (temp->title == title) return true;
        temp = temp->next;
    }
    return false;
}

void updateSong(string title, string newTitle, string newArtist, int newDuration)
{
    Song* temp = head;
    while (temp != nullptr) {
        if (temp->title == title) {
            temp->title = newTitle;
            temp->artist = newArtist;
            temp->duration = newDuration;
            return;
        }
        temp = temp->next;
    }
}

void displayPlaylist() {
    Song* temp = head;
    while (temp != nullptr) {
        cout << temp->title << " - " << temp->artist << " (" << temp->duration <<
"s)" << endl;
        temp = temp->next;
    }
}

```

```

}
void arrangeSongs() {
    if (head == nullptr) return;
    bool swapped;
    do {
        swapped = false;
        Song* current = head;
        while (current->next != nullptr) {
            if (current->title > current->next->title) {
                swap(current->title, current->next->title);
                swap(current->artist, current->next->artist);
                swap(current->duration, current->next->duration);
                swapped = true;
            }
            current = current->next;
        }
    } while (swapped);
}
};

class CircularLinkedList {
private:
    Song* head;
public:
    CircularLinkedList() {
        head = nullptr;
    }
    void addSong(string title, string artist, int duration) {
        Song* newSong = new Song(title, artist, duration);
        if (head == nullptr) {
            head = newSong;
            newSong->next = head;
        } else {
            Song* temp = head;
            while (temp->next != head) {
                temp = temp->next;
            }
            temp->next = newSong;
            newSong->next = head;
        }
    }
    void deleteSong(string title) {
        if (head == nullptr) return;
        if (head->title == title && head->next == head) {
            delete head;
            head = nullptr;
            return;
        }
        Song* temp = head;
        Song* prev = nullptr;
        if (head->title == title) {
            while (temp->next != head) {
                temp = temp->next;
            }

```

```

    }
    temp->next = head->next;
    Song* toDelete = head;
    head = head->next;
    delete toDelete;
    return;
}
while (temp->next != head && temp->next->title != title) {
    temp = temp->next;
}
if (temp->next->title == title) {
    Song* toDelete = temp->next;
    temp->next = temp->next->next;
    delete toDelete;
}
}
bool searchSong(string title) {
    if (head == nullptr) return false;
    Song* temp = head;
    do {
        if (temp->title == title) return true;
        temp = temp->next;
    } while (temp != head);
    return false;
}
void updateSong(string title, string newTitle, string newArtist, int newDuration)
{
    if (head == nullptr) return;
    Song* temp = head;
    do {
        if (temp->title == title) {
            temp->title = newTitle;
            temp->artist = newArtist;
            temp->duration = newDuration;
            return;
        }
        temp = temp->next;
    } while (temp != head);
}
void displayPlaylist() {
    if (head == nullptr) return;
    Song* temp = head;
    do {
        cout << temp->title << " - " << temp->artist << " (" << temp->duration <<
"s)" << endl;
        temp = temp->next;
    } while (temp != head);
}
void arrangeSongs() {
    if (head == nullptr) return;
    bool swapped;
    do {

```

```

        swapped = false;
        Song* current = head;
        do {
            if (current->next != head && current->title > current->next->title) {
                swap(current->title, current->next->title);
                swap(current->artist, current->next->artist);
                swap(current->duration, current->next->duration);
                swapped = true;
            }
            current = current->next;
        } while (current->next != head);
    } while (swapped);
}

};

int main() {
    SinglyLinkedList sll;
    DoublyLinkedList dll;
    CircularLinkedList cll;
    int mainChoice, listChoice;
    string title, artist, newTitle;
    int duration, newDuration;
    do {
        cout << "\nMain Menu:\n";
        cout << "1. Singly Linked Playlist\n";
        cout << "2. Doubly Linked Playlist\n";
        cout << "3. Circular Linked Playlist\n";
        cout << "4. Exit\n";
        cout << "Enter choice: ";
        cin >> mainChoice;
        if (mainChoice == 4) break;
        do {
            cout << "\n1. Add Song\n";
            cout << "2. Delete Song\n";
            cout << "3. Search Song\n";
            cout << "4. Update Song\n";
            cout << "5. Display Playlist\n";
            cout << "6. Arrange Songs\n";
            cout << "7. Exit to Main Menu\n";
            cout << "Enter choice: ";
            cin >> listChoice;
            switch(listChoice) {
                case 1:
                    cout << "Enter song title: ";
                    cin.ignore();
                    getline(cin, title);
                    cout << "Enter artist: ";
                    getline(cin, artist);
                    cout << "Enter duration (seconds): ";
                    cin >> duration;
                    if (mainChoice == 1) sll.addSong(title, artist, duration);
                    else if (mainChoice == 2) dll.addSong(title, artist, duration);

```

```

else if (mainChoice == 3) cll.addSong(title, artist, duration);
break;
case 2:
    cout << "Enter song title to delete: ";
    cin.ignore();
    getline(cin, title);
    if (mainChoice == 1) sll.deleteSong(title);
    else if (mainChoice == 2) dll.deleteSong(title);
    else if (mainChoice == 3) cll.deleteSong(title);
    break;
case 3:
    cout << "Enter song title to search: ";
    cin.ignore();
    getline(cin, title);
    if (mainChoice == 1)
        cout << (sll.searchSong(title) ? "Found" : "Not Found") <<
endl;

    else if (mainChoice == 2)
        cout << (dll.searchSong(title) ? "Found" : "Not Found") <<
endl;

    else if (mainChoice == 3)
        cout << (cll.searchSong(title) ? "Found" : "Not Found") <<
endl;

    break;
case 4:
    cout << "Enter song title to update: ";
    cin.ignore();
    getline(cin, title);
    cout << "Enter new title: ";
    getline(cin, newTitle);
    cout << "Enter new artist: ";
    getline(cin, artist);
    cout << "Enter new duration: ";
    cin >> newDuration;
    if (mainChoice == 1) sll.updateSong(title, newTitle, artist,
newDuration);
    else if (mainChoice == 2) dll.updateSong(title, newTitle, artist,
newDuration);
    else if (mainChoice == 3) cll.updateSong(title, newTitle, artist,
newDuration);

    break;
case 5:
    if (mainChoice == 1) sll.displayPlaylist();
    else if (mainChoice == 2) dll.displayPlaylist();
    else if (mainChoice == 3) cll.displayPlaylist();
    break;
case 6:
    if (mainChoice == 1) sll.arrangeSongs();
    else if (mainChoice == 2) dll.arrangeSongs();
    else if (mainChoice == 3) cll.arrangeSongs();
    break;
case 7:

```

```

        break;
    default:
        cout << "Invalid choice!" << endl;
    }
    } while (listChoice != 7);
} while (mainChoice != 4);
return 0;
}

```

➤ OUTPUT –

1. Singly LL

Main Menu:

1. Singly Linked Playlist
2. Doubly Linked Playlist
3. Circular Linked Playlist
4. Exit

Enter choice: 1

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 1

Enter song title: Ilahi

Enter artist: Pritam

Enter duration (seconds): 240

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 1

Enter song title: Jiya Laage Na

Enter artist: Sona M

Enter duration (seconds): 120

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 5

Ilahi - Pritam (240s)

Jiya Laage Na - Sona M (120s)

1. Add Song
2. Delete Song

3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu
Enter choice: 3
Enter song title to search: Ilahi
Found

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu
Enter choice: 4
Enter song title to update: Jiya Laage Na
Enter new title: Jiya Laage Na New
Enter new artist: Sona M.
Enter new duration: 180

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu
Enter choice: 5
Ilahi - Pritam (240s)
Jiya Laage Na New - Sona M. (180s)

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu
Enter choice: 6
Sorted

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu
Enter choice: 5
Ilahi - Pritam (240s)
Jiya Laage Na New - Sona M. (180s)

2. Doubly LL

Main Menu:

1. Singly Linked Playlist
2. Doubly Linked Playlist
3. Circular Linked Playlist
4. Exit

Enter choice: 2

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 1

Enter song title: Naadan Parinde

Enter artist: AR Rahman

Enter duration (seconds): 150

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 1

Enter song title: Vaaroon

Enter artist: Anand B.

Enter duration (seconds): 360

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 3

Enter song title to search: Vaaroon

Found

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 4

Enter song title to update: Nadaan Parinde

Enter new title: Naadan Pareende

Enter new artist: AR Rahman

Enter new duration: 160

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 2
Enter song title to delete: Naadan Parinde

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 1
Enter song title: Dooron Dooron
Enter artist: ABC
Enter duration (seconds): 240

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 6

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 5
Dooron Dooron - ABC (240s)
Vaaroon - Anand B. (360s)

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu

Enter choice: 7

3. Circular LL

Main Menu:

1. Singly Linked Playlist
2. Doubly Linked Playlist

3. Circular Linked Playlist

4. Exit

Enter choice: 3

1. Add Song

2. Delete Song

3. Search Song

4. Update Song

5. Display Playlist

6. Arrange Songs

7. Exit to Main Menu

Enter choice: 1

Enter song title: Arz Kiya Hain

Enter artist: Anuv J

Enter duration (seconds): 230

1. Add Song

2. Delete Song

3. Search Song

4. Update Song

5. Display Playlist

6. Arrange Songs

7. Exit to Main Menu

Enter choice: 1

Enter song title: Wavy

Enter artist: Karan A

Enter duration (seconds): 270

1. Add Song

2. Delete Song

3. Search Song

4. Update Song

5. Display Playlist

6. Arrange Songs

7. Exit to Main Menu

Enter choice: 3

Enter song title to search: Wavy

Found

1. Add Song

2. Delete Song

3. Search Song

4. Update Song

5. Display Playlist

6. Arrange Songs

7. Exit to Main Menu

Enter choice: 4

Enter song title to update: Wavy

Enter new title: Wavy

Enter new artist: Karan Aujla

Enter new duration: 290

1. Add Song

2. Delete Song

3. Search Song

4. Update Song

5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu
Enter choice: 5
Arz Kiya Hain - Anuv J (230s)
Wavy - Karan Aujla (290s)

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu
Enter choice: 6

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu
Enter choice: 2
Enter song title to delete: Wavy

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu
Enter choice: 5
Arz Kiya Hain - Anuv J (230s)

1. Add Song
2. Delete Song
3. Search Song
4. Update Song
5. Display Playlist
6. Arrange Songs
7. Exit to Main Menu
Enter choice: 7

➤ **CONCLUSION –**

This lab assignment implementation demonstrates use of different type of linked lists for creating real world applications like music playlist.